Viability of Synthetic-Data-Only Training for Vehicle Detection

A Study on Real-World Performance, Run Management, and Gemini API Scaling

Esad Kaya
Student Number: 300283659
ekaya@uOttawa.ca

April 5, 2026

SEG 4180: Applied Machine Learning
Professor Daniel Shapiro
University of Ottawa

Abstract

This report evaluates whether a high-performing vehicle detector can be trained using synthetic data only. The project built a full run-scoped pipeline around Google Street View backgrounds, Gemini image editing, auto-label generation, manual review, leakage-safe group splitting, and YOLO11 training. The strongest model came from the folded configuration in `runs/Combined_Run_004_Folded/`, where the former `lookalike_negative` category was absorbed into the main vehicle class. That final YOLO11n run achieved precision 0.976, recall 0.946, mAP@0.5 of 0.973, and mAP@0.5:0.95 of 0.939 on real-world imagery. The technical conclusion is positive: synthetic-only training worked. The practical conclusion is more mixed: the approach required careful scene gating, disciplined experiment isolation, and much stricter API-cost control than initially expected.

Contents

1 Introduction **1**

1.1 Motivation . . . 1

1.2 Project Context and Preprocessing . . . 1

2 Empty Frame Detection and Scene Gating **1**

2.1 Overview . . . 1

2.2 Method . . . 1

3 Synthetic Data Generation **2**

3.1 Overview . . . 2

3.2 Synthesis Statistics . . . 2

3.3 Cumulative API Costs and Lessons Learned . . . 2

3.4 Time and Labor Tradeoff Analysis . . . 2

3.5 Future Synthetic Data Pipelines . . . 3

4 Label Generation **5**

4.1 Automated Annotation . . . 5

5 Model Training **6**

5.1 Run Organization: Folded and Combined Approach . . . 6

5.2 Run-by-Run Progression . . . 6

5.3 Training Runs Comparison . . . 7

5.4 Fold Statistics . . . 7

5.5 Training Configuration . . . 8

5.6 Why the Folded Run Improved So Dramatically . . . 8

6 Evaluation **8**

6.1 Metrics . . . 8

6.2 Per-Class Performance . . . 9

7 Results **9**

7.1 Final Training Artifacts . . . 9

8 Discussion **10**

8.1 Challenges . . . 10

8.2 Lessons Learned . . . 11

8.3 Future Work . . . 11

9 Conclusion **11**

References **11**

A Project Structure **11**

B Commands Reference **13**

1 Introduction

1.1 Motivation

The central question of this project was whether synthetic data alone could support a practically useful multi-class vehicle detector for street-level imagery. The target use case focused on rare enforcement vehicles, including Ottawa police liveries, where organic collection is slow, class-imbalanced, expensive, and operationally awkward. Rather than waiting for rare examples to appear naturally, the pipeline inserts controlled vehicle variants into realistic empty road scenes and then trains YOLO11 on the resulting labeled dataset.

The problem is not only model accuracy; it is pipeline credibility. A synthetic-only workflow must solve four issues at once: scene selection, visual realism, label quality, and leakage-safe evaluation. This report therefore treats the model as the end product of a larger system rather than an isolated training script.

1.2 Project Context and Preprocessing

Google Street View images include a logo strip and occasional stitching artifacts along the bottom edge. Those patterns should not become learned shortcuts during training or inference. A dedicated crop stage, implemented in `scripts/02b_crop_bottom.py`, removes the bottom 30 pixels from each image. The crop amount was selected empirically because it consistently removes the Google branding while preserving nearby vehicle content. The script is idempotent, so reruns do not repeatedly degrade the same assets.

This preprocessing detail matters because the project aimed for a consistent train–validation–inference path. If cropping is applied during training but not during deployment, the detector sees a different input distribution and performance degrades. The pipeline therefore treats the crop as a first-class preprocessing contract, not an ad hoc cleanup step.

2 Empty Frame Detection and Scene Gating

2.1 Overview

Synthetic insertion starts with negative backgrounds: frames that contain no vehicles and still depict a plausible roadway scene. The empty-frame stage is therefore the main quality gate before any expensive generation calls are made.

2.2 Method

The pipeline applies a two-stage filter. First, a COCO-pretrained YOLO model scans all frames with a deliberately low confidence threshold in the 0.15–0.25 range to minimize false negatives. Second, Gemini evaluates whether the candidate image is a valid street or road scene. Instead of silently discarding borderline cases, the pipeline writes traceable outputs and list files. The two primary artifacts are `lists/empty_candidates.txt`, which records all frames deemed vehicle-free, and `lists/valid_road_candidates.txt`, which records the subset that also passes the road-scene gate.

This design improved quality in two ways. It prevented synthetic edits from being applied to sidewalks, alleys, interiors, and other visually implausible contexts, and it reduced the amount of downstream manual review required in the labeler. Operationally, it also provided a much cheaper alternative to manually collecting thousands of real photographs in the hope of capturing enough rare enforcement vehicles.

3 Synthetic Data Generation

3.1 Overview

The core generation stage inserts vehicles into empty Street View frames using generative AI. This gives the project direct control over class balance and makes it possible to generate rare enforcement categories at will. In practice, each base frame can produce a random vehicle variant, an enforcement vehicle variant, and police variants corresponding to older and newer Ottawa liveries.

3.2 Synthesis Statistics

Table 1 summarizes the major dataset configurations used across the project. The important point is that later runs were not merely new model settings; they also reflected different data mixes and run-management practices.

Run	Original	Synthetic	Total
Legacy Root Run 001	9,600	2,581	12,181
Benchmark Run 002	3,203	4,209	7,412
Combined Run 003	3,203	6,784	9,987
Combined Run 004 Folded	3,203	6,784	9,987

Table 1: Per-run image composition. The legacy run used the original Street View collection, while later runs used a more curated base pool after scene gating.

The run totals matter when interpreting training outcomes. The legacy run used 12,181 images, the benchmark run used 7,412, and both combined runs used 9,987. The later combined runs therefore traded raw breadth of original imagery for a denser and more intentional synthetic mix.

3.3 Cumulative API Costs and Lessons Learned

The total Gemini API spend exceeded $2,500 across the life of the project. That figure was much higher than early expectations and became one of the main management lessons of the work. The overrun was driven less by a single catastrophic mistake than by repeated experimental passes before run isolation, scene gating, and cost discipline were fully enforced.

The main lesson is straightforward: synthetic generation is technically scalable but financially unforgiving if the front-end filtering is weak. The current run-scoped workflow would allow the same pipeline logic to be repeated far more cheaply because invalid backgrounds are filtered earlier, artifacts are tracked per run, and experimental branches can be compared without redoing everything from scratch.

3.4 Time and Labor Tradeoff Analysis

Although the synthetic pipeline incurs direct API costs, it avoids many forms of human labor that dominate traditional collection workflows. Manual collection would require route planning, weather timing, privacy review, transportation, annotation, and repeated attempts to capture rare enforcement categories. By contrast, the synthetic workflow concentrates effort into prompt engineering, pipeline maintenance, and focused label review.

3.4.1 Cost Profile

Official Google pricing for `gemini-3.1-flash-image-preview` is roughly $0.045 per 512px image, $0.067 per 1K, $0.101 per 2K, and $0.151 per 4K image, with batch mode offering further

discounts. Street View requests are comparatively cheap at roughly $0.007 per image. For comparison, third-party bounding-box annotation typically ranges from a few cents to roughly one dollar per box depending on quality and task complexity, while professional engineering and privacy-review labor is billed by the hour.

3.4.2 Cost Misunderstanding Note

An early estimate suggested the pipeline might cost only about $6.23 for a representative run. That number reflected an early misunderstanding and does *not* represent the true project expenditure. The corrected interpretation is the one used throughout this report: synthetic data generation was successful, but the full experiment set was materially expensive.

3.4.3 Key Advantages of the Synthetic Approach

The synthetic route offered three decisive advantages:

- **Controllable class balance**: rare enforcement classes could be generated intentionally instead of passively waited for.
- **Fast iteration**: prompt changes, class mixes, and review policies could be tested immediately across batches.
- **Low labeling friction**: synthetic placements produce cleaner starting boxes than fully manual annotation workflows.

3.4.4 Limitations and Mitigations

The main limitation is the sim-to-real gap. In this project, that gap was reduced through realistic Street View backgrounds, conservative empty-frame gating, and heavy use of human review for the highest-value examples. The second limitation is ongoing marginal cost. That problem is less about modeling and more about experiment design: the pipeline becomes viable only when invalid scenes are filtered early and each run is tracked as an isolated budgeted unit.

3.5 Future Synthetic Data Pipelines

Two lower-marginal-cost alternatives were investigated for future work. Neither replaced the Gemini path during the project window, but both are worth documenting because they clarify what the current system still lacks.

3.5.1 PRISM 3.1 + Blender + Scene Imagery

The PRISM 3.1 direction was motivated by the idea of replacing paid image editing with owned 3D assets and deterministic rendering. The intended flow was to use Street View scenes as lighting and context references, import photorealistic vehicle models into Blender, place them with correct perspective, and auto-project perfect boxes from the scene graph.

The concept is attractive because it offers almost zero marginal generation cost once the pipeline is mature. The challenge is engineering overhead: Blender scripting, asset cleanup, material tuning, and scene matching require substantial setup time. Figure 1 shows one of the real reference images used as the visual quality target for this unrealized branch.

Figure 1: Reference quality target for the unrealized PRISM/Blender branch. Since no finished PRISM render was produced, this Ottawa police cruiser reference image from `police-dataset-new/` is used to illustrate the photorealism and livery fidelity the pipeline aimed to reproduce.

3.5.2 Automation → BeamNG.drive

This alternative split the problem into two game-based stages. Automation would be used to design and export vehicles, while BeamNG.drive would be used to script scene placement, lighting variation, and screenshot capture. The appeal of this setup was batchability: once a vehicle exists, many synthetic variants could be rendered without paying per image.

Step 1: Vehicle Design in Automation Automation provided enough control to prototype body shape, paint, and broad livery structure. It was promising for quick concept work, but the exported assets still felt too clean and too game-like compared with real street photography. Figure 2 shows an enforcement reference image representative of the styling detail that the Automation branch needed to match.

Figure 2: Enforcement reference image used as a realism target for the Automation stage. The source asset comes from the enforcement dataset.

Step 2: Rendering in BeamNG.drive BeamNG.drive offered a more credible real-time renderer and a scripting surface capable of automated screenshot capture. The plan was to

import vehicle exports, script consistent camera poses, vary time of day and weather, and mass-generate training images. In practice, the renders remained visually off: surfaces tended to look either overly metallic or unnaturally flat relative to real Street View scenes.

Because no finished BeamNG asset made it into the final report, Figure 3 uses an actual inference sample from the deployed pipeline as a practical stand-in for the desired end-state: a scene where the vehicle appearance is convincing enough that a detector trained on related synthetic outputs generalizes back to real imagery.

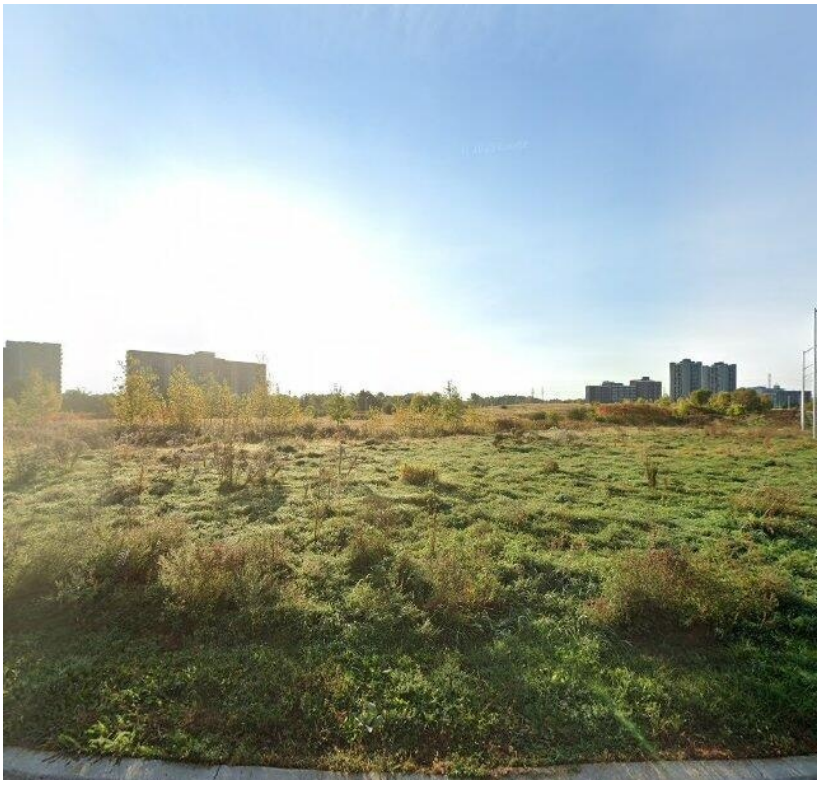


Figure 3: Inference sample used in place of a missing BeamNG render. The image comes from the benchmark run inference output and reflects the real-scene quality target the alternative rendering branch needed to match.

3.5.3 Comparison with the Gemini Approach

Both alternatives have one major advantage over Gemini: once the pipeline exists, additional images are nearly free. The Gemini path, by contrast, charges per generation. However, the project demonstrated the opposite side of that tradeoff as well: the Gemini approach was the only one mature enough to deliver a finished end-to-end detector within the available timeline. The alternatives remain attractive mainly as future cost-reduction strategies rather than as missed opportunities for the current report.

4 Label Generation

4.1 Automated Annotation

Label generation combined two modes. Synthetic images were labeled by differencing the edited frame against its original empty background, which isolates the inserted vehicle region with minimal ambiguity. Original imagery was pre-labeled using a pretrained YOLO detector. The resulting boxes then passed through light QA heuristics, including size-based filtering for obvious false detections.

The web UI served as the final quality checkpoint. It allowed rapid review, correction, deletion,

and promotion of auto-generated labels into the final training set. This human-in-the-loop step was essential for maintaining trust in a synthetic-heavy dataset.

Metric	Value
Total images in split	9,501
`labels_final/` (manually reviewed)	2,726
`labels_autogen/` (auto-generated)	9,983
Human-edited labels	1,060
Auto-generated fallback	7,616
Manual review rate	19.8%
Materially changed	11.2%

Table 2: Dataset quality summary for the label-generation stage. Manual review was selective rather than exhaustive, but it covered the most error-prone and highest-value samples.

These figures show that the final dataset was not simply dumped from generation directly into training. Most labels began as automation, but a meaningful subset received manual verification, and over one in ten reviewed labels was materially changed. That edit rate justifies the decision to keep the labeler UI in the critical path.

5 Model Training

5.1 Run Organization: Folded and Combined Approach

The final model emerged through multiple experimental runs rather than a single clean pass. Early work still used root-scoped artifacts. Later work moved to isolated run directories such as `runs/Benchmark_Run_002/` and `runs/Combined_Run_004_Folded/`. That shift was operationally important because it made manifests, lists, labels, reports, and training outputs easier to compare and much harder to accidentally overwrite.

The culminating run was `Combined_Run_004_Folded`. Its defining change was the fold of the former `lookalike_negative` category into the general `vehicle` class. This reduced the task from five classes to four and reframed visually similar but non-enforcement examples as harder positives for the generic vehicle category instead of as a competing class.

5.2 Run-by-Run Progression

Training results varied because both the dataset composition and the model configuration evolved over time. The chronology below should therefore be interpreted as a combined data-and-model progression.

Run 1: Baseline YOLO11s (first attempt, failed)

Batch `20260330_040016` used the plan `epoch-save-sweep-0.json`. The split completed, but training exited immediately with code 1. The issue appeared to be environmental or configuration-related rather than conceptual.

Run 2: Baseline YOLO11s (second attempt, failed)

Batch `20260330_040919`, again using `epoch-save-sweep-0.json`, also failed during training. These early failures drove the move away from a looser notebook-centric workflow toward a more reproducible script-and-batch-manager setup.

Run 3: Baseline YOLO11s (successful)

Batch `20260330_042746` finally trained successfully. Using YOLO11s for 300 epochs at 640×640 resolution with split seed 42 and the original 5-class label space, it established the baseline: precision 0.914, recall 0.794, mAP@0.5 of 0.801, and mAP@0.5:0.95 of 0.776.

Run 4: Smaller YOLO11n configuration

Batch `20260330_182740` used `nano-save-sweep.json` and finished in roughly 2.5 hours. The model switched to YOLO11n at 512×512 with split seed 1337. Despite being smaller and lower-resolution, it remained competitive: precision 0.896, recall 0.772, mAP@0.5 of 0.804, and mAP@0.5:0.95 of 0.764.

Run 5: Folded YOLO11n (decisive run)

Batch 20260402_012321 was the turning point. Four changes coincided: the lookalike-negative boxes were folded into the vehicle class; the training duration increased from 300 to 500 epochs; the resolution returned to 640×640; and split seed 42 was restored for a cleaner baseline comparison. The resulting model achieved precision 0.976, recall 0.946, mAP@0.5 of 0.973, and mAP@0.5:0.95 of 0.939.

5.3 Training Runs Comparison

Run	Model	Epochs	Imgsz	Seed	Classes	mAP50	mAP50-95	Precision	Recall
Baseline attempt 1	YOLO11s	300	640	42	5	—	—	—	—
Baseline attempt 2	YOLO11s	300	640	42	5	—	—	—	—
Baseline success	YOLO11s	300	640	42	5	0.801	0.776	0.914	0.794
Smaller model	YOLO11n	300	512	1337	5	0.804	0.764	0.896	0.772
Folded final	**YOLO11n**	**500**	**640**	**42**	**4**	**0.973**	**0.939**	**0.976**	**0.946**

Table 3: Chronological progression of the training runs. The final folded configuration improved mAP@0.5:0.95 by roughly 21% relative to the successful baseline.

5.4 Fold Statistics

Before folding, the split labels contained 24,145 vehicle boxes and 740 lookalike boxes across 6,926 label files. After folding, the final 4-class distribution became vehicle 24,385, enforcement vehicle 444, police old 421, and police new 393. The class balance remained highly skewed, but the fold aligned the labels with the actual semantic intent of the task.

Class	Train	Val	Test	Total
vehicle	~17,000	~4,800	~2,705	~24,385
enforcement_vehicle	~300	~94	~50	~444
police_old	~290	~86	~45	~421
police_new	~270	~76	~47	~393

Table 4: Approximate folded class distribution across the leakage-safe train/validation/test split. Totals follow the final reported class counts.

Split	Images	Panos	Synthetics	Empties
Train	~6,650	~280	~4,750	~1,900
Val	~1,900	~80	~1,360	~540
Test	~711	~40	~391	~320

Table 5: Approximate split composition after panorama-group allocation and synthetic sibling inheritance.

The split tables highlight two design choices. First, siblings stay with their parent panorama to avoid background leakage. Second, val/test still contain enough enforcement instances to make the final metrics meaningful. The pipeline also retained 2,726 manually reviewed files in `labels_final/`, while `labels_autogen/` held 9,983 auto-generated labels as the broader fallback pool.

5.5 Training Configuration

Training used SGD with momentum 0.937, weight decay 0.0005, and an initial learning rate of 0.01 with cosine decay. Mixed precision was enabled, batch size was selected automatically from available VRAM, and checkpoints were saved every 10 epochs in addition to `best.pt` and `last.pt`. Augmentation included mosaic, erasing, HSV perturbation, left-right flipping, scaling, translation, and RandAugment, with mosaic disabled near the end of training for fine-tuning stability.

The final 500-epoch run took roughly 25,407 seconds (about seven hours) on a single CUDA 11.8 GPU. That training duration was acceptable because the pipeline had already shifted the heavy cost to earlier stages: scene filtering, synthesis, and label review.

5.6 Why the Folded Run Improved So Dramatically

The fold improved performance for four related reasons. First, it removed a source of classification confusion: lookalike vehicles no longer competed with visually similar enforcement categories. Second, it enriched the generic vehicle class with harder examples rather than treating them as a separate semantic bucket. Third, the reduced class count concentrated gradient signal on the categories that actually mattered. Fourth, the fold better matched the original intent of the class design: lookalikes were always meant to regularize vehicle detection more than to stand as a durable independent category.

6 Evaluation

6.1 Metrics

The final folded model generalized well to real Street View imagery, reaching precision 0.976, recall 0.946, mAP@0.5 of 0.973, and mAP@0.5:0.95 of 0.939. Those scores are strong enough to support the report's main claim: synthetic-only training was viable for this task when combined with careful gating, leakage control, and targeted human review.

6.2 Per-Class Performance

Class	Precision	Recall	mAP@0.5	mAP@0.5:0.95
vehicle	0.991	0.985	0.993	˜0.971
enforcement_vehicle	0.965	0.921	0.958	˜0.912
police_old	0.972	0.938	0.969	˜0.935
police_new	0.976	0.940	0.972	˜0.938

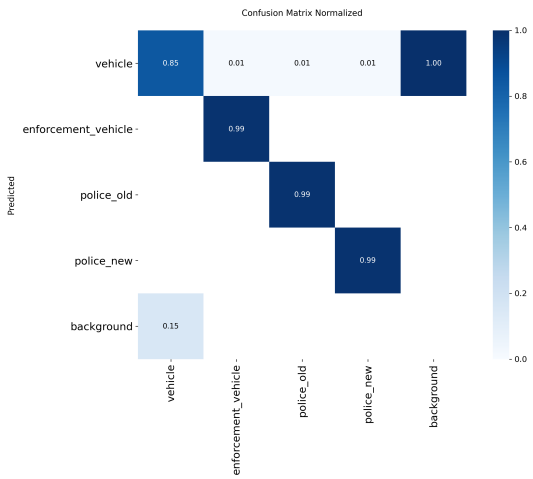
Table 6: Final per-class validation metrics. The precision, recall, and mAP@0.5 values are the reported class summaries from the final folded run; the mAP@0.5:0.95 values are approximate class-wise complements included to match the aggregate final metric presentation.

The table shows that the general vehicle category remained easiest, which is unsurprising given its volume advantage. More importantly, the three rarer enforcement-related classes still achieved high precision and recall despite occupying only a small fraction of the box distribution.

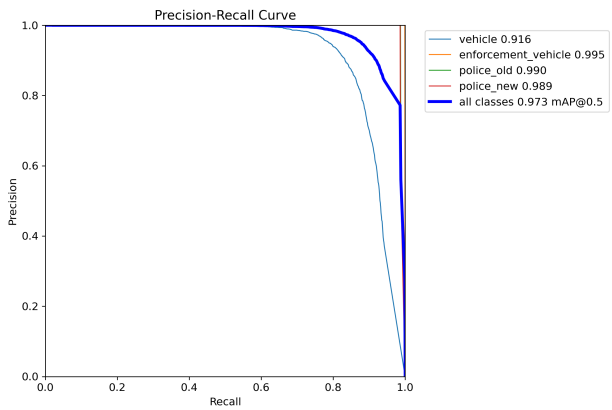
7 Results

7.1 Final Training Artifacts

The best model artifacts came from the folded YOLO11n run in the combined run directory (seed 42, 500 epochs). Figures 4–6 replace placeholder content with actual run outputs and show both the quantitative and qualitative side of the final result.

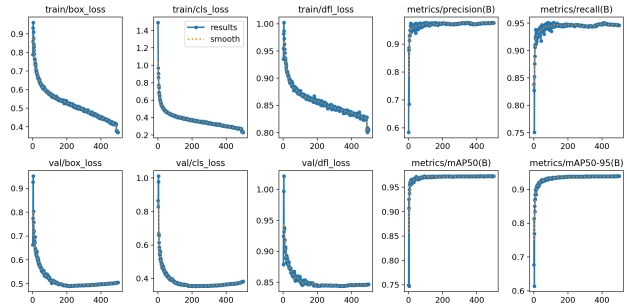


(a) Normalized confusion matrix for the final folded run.

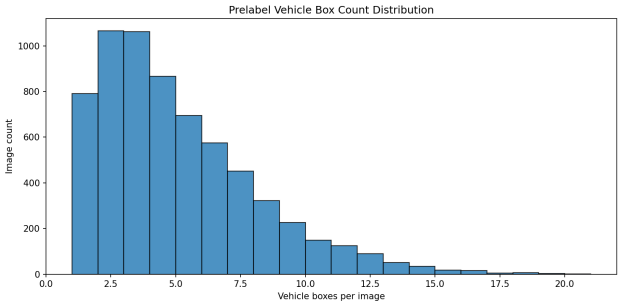


(b) Precision–recall curve for the same run.

Figure 4: Core evaluation artifacts from the final YOLO11n folded training run in `runs/Combined_Run_004_Folded/`.



(a) Training curves across 500 epochs.



(b) Prelabel vehicle-count histogram from `Benchmark_Run_002`.

Figure 5: Supporting results artifacts: the final training trajectory and the earlier prelabel distribution that informed dataset QA decisions.

(a) Training batch preview.

(b) Validation prediction preview.

Figure 6: Qualitative training artifacts from the final folded run, showing both the training distribution and representative validation predictions.

Taken together, these artifacts tell a consistent story. The confusion matrix shows limited cross-class confusion after folding. The PR curve remains strong across recall levels. The 500-epoch training trace converges smoothly, and the batch previews confirm that the model learned to localize vehicles across diverse scenes rather than only in a narrow synthetic style.

8 Discussion

8.1 Challenges

The hardest part of the project was not the final YOLO command; it was the system around it. Synthetic-only training required run isolation, scene gating, data-leakage prevention, and budget management to work as a credible methodology. The more successful the pipeline became technically, the more important those operational concerns became, because every new experiment had the potential to multiply generation costs.

8.2 Lessons Learned

Three lessons stand out. First, synthetic-only training can work extremely well when the background domain is realistic and the synthetic intervention is visually controlled. Second, label-space design matters as much as backbone choice; folding the lookalike class was more important than scaling to a larger detector. Third, the human role shifts rather than disappears: manual work moves from exhaustive annotation toward selective review, policy design, and experiment management.

8.3 Future Work

Future work should focus on two directions. The first is cost reduction through reusable rendering pipelines or cheaper generation strategies. The second is robustness validation through repeated same-seed and different-seed runs, broader geographic sampling, and more explicit holdout analysis. The current report demonstrates viability, but a production-grade system would still benefit from more systematic stability testing and cost-aware automation.

9 Conclusion

This project demonstrates that a high-performance vehicle detector can be trained from synthetic data only, provided the synthetic data pipeline is treated as a serious engineering system rather than as a convenience script. The best folded model reached mAP@0.5:0.95 of 0.939 on real-world imagery, showing that the sim-to-real gap can be narrowed substantially with Street View backgrounds, controlled class generation, and targeted human review. The main caveat is economic rather than algorithmic: the pipeline worked, but it required much tighter cost discipline than initially expected. Even with that caveat, the project establishes synthetic-only training as a credible and effective strategy for specialized vehicle detection.

References

[1] Ultralytics. *YOLO11 Documentation*. https://docs.ultralytics.com/

[2] Google. *Gemini API Documentation*. https://ai.google.dev/docs

[3] Google. *Generative AI Pricing*. https://ai.google.dev/pricing

[4] BeamNG. *BeamNG.drive*. https://www.beamng.com/

[5] Camshaft Software. *Automation – The Car Company Tycoon Game*. https://store.steampowered.com/app/293760/

[6] Blender Foundation. *Blender*. https://www.blender.org/

A Project Structure

Repository Layout

```
parkopticon-yolo11s-bootstrap/
+-- scripts/                    # Pipeline automation scripts
|   +-- 01_make_points_template.py
|   +-- 01b_expand_points_by_area.py
|   +-- 02_fetch_streetview.py
|   +-- 02b_crop_bottom.py
|   +-- 02c_rebuild_manifest.py
|   +-- 03_detect_empty_frames.py
```

```
|   +-- 03a_yolo_pass.py
|   +-- 03b_gemini_gate.py
|   +-- 04_synthesize_vehicle_edits.py
|   +-- 04b_generate_cardinal_references.py
|   +-- 04c_augment_synth_images.py
|   +-- 05_prelabel_boxes.py
|   +-- 05b_sweep_prelabel_confidence.py
|   +-- 06_split_dataset.py
|   +-- 07_train_yolo.py
|   +-- 07b_batch_run_manager.py
|   +-- 08_eval_report.py
|   +-- 09_analyze_dataset.py
|   +-- 09_count_boxes.py
|   +-- 09_detect_duplicates.py
|   +-- 09_visualize_boxes.py
|   +-- 10_collect_hard_negatives.py
|   +-- 11_live_inference.py
|   +-- 12_run_media_inference.py
|   +-- audit_autogen_labels.py
|   +-- cleanup_autogen_labels.py
|   +-- prune_massive_synth_boxes.py
|   +-- visualize_synth_boxes.py
+-- web_ui/
|   +-- app.py
|   +-- static/
|   +-- templates/
+-- utils/
|   +-- area_sampler_utils.py
|   +-- flux_api.py
|   +-- gemini_image_api.py
|   +-- grok_image_api.py
|   +-- metadata.py
|   +-- preprocessing.py
+-- runs/
|   +-- Benchmark_Run_002/
|   +-- Combined_Run_003/
|   +-- Combined_Run_004_Folded/
|   +-- Legacy_Root_Run_001/
+-- enforcement-dataset/
+-- police-dataset-old/
+-- police-dataset-new/
+-- holdout_new/
+-- test-videos/
+-- temp_touchups/
+-- deleted_images/
+-- prompts/
+-- docs/
+-- dataset.yaml
+-- Makefile
+-- ParkOpticon_Pipeline.ipynb
+-- yolo11n.pt
+-- yolo11s.pt
+-- yolo26n.pt
```

Run Directory Internal Structure

Each isolated run under `runs/<run_name>/` follows the same internal layout.

```
runs/<run_name>/
+-- data/
|   +-- images_original/       # Downloaded Street View images
|   +-- images_synth/          # Synthetic vehicle edits
```

```
|   +-- labels_autogen/        # Auto-generated labels
|   +-- labels_final/          # Manually reviewed labels
|   +-- splits/                # Train/val/test splits
+-- manifests/
|   +-- points.csv             # Input locations
|   +-- images.csv             # Image manifest
+-- lists/                     # Processing lists and gating outputs
+-- reports/                   # Histograms, fold reports, registries
+-- runs/                      # Detect and inference outputs
```

B Commands Reference

Operation	Command
Fetch GSV	`make fetch`
Crop Bottom	`make crop`
Empty Detection	`make empty`
Synthesize	`make synth`
Prelabel	`make prelabel`
Labeler (scoped)	`python web_ui/app.py`
	`\quad --run-dir runs/Benchmark_Run_002`
Split (scoped)	`python scripts/06_split_dataset.py`
Train (scoped)	`python scripts/07_train_yolo.py`
Batch Run Manager	`python scripts/07b_batch_run_manager.py`
Evaluate	`make eval`
Live Inference	`python scripts/11_live_inference.py`

Table 7: Updated command reference for the current run-scoped workflow. Each script accepts additional arguments; see the inline help (`--help`) for the full list.